

A Look at Key CNN Architectures

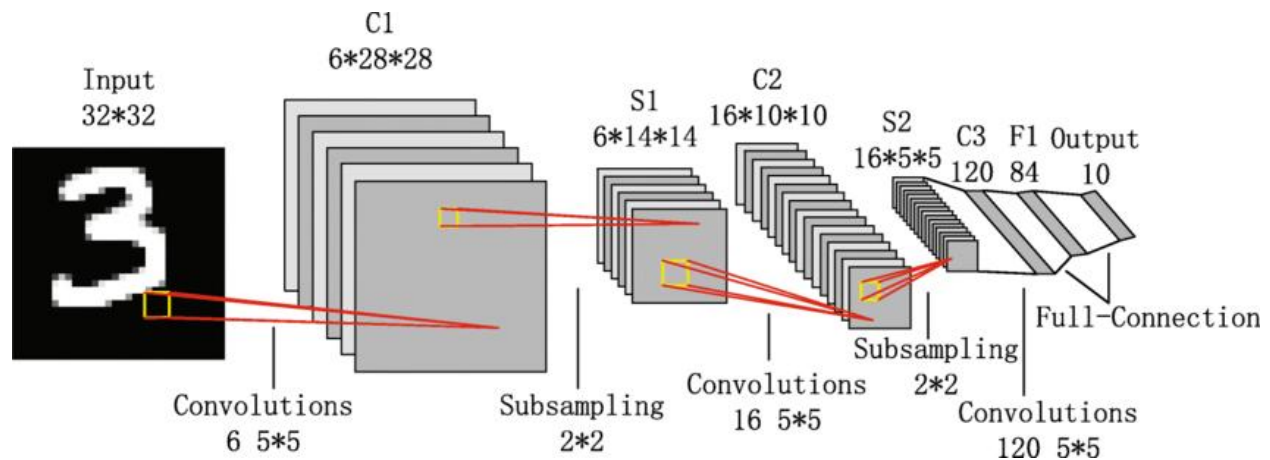
The field of computer vision has been revolutionized by Convolutional Neural Networks (CNNs). Here is a breakdown of several foundational and influential CNN architectures, with a focus on their unique contributions.

LeNet

LeNet was one of the first and most pioneering CNN architectures, introduced by Yann LeCun in the late 1990s. Its primary application was recognizing handwritten digits. While it seems simple today, it established the core building blocks that are still used in modern CNNs.

Key Contributions:

- **Convolutional Layers:** LeNet was one of the first to successfully use convolutional layers to automatically extract features from images. This was a paradigm shift from traditional methods that required manual feature engineering.
- **Pooling Layers:** It introduced pooling (subsampling) layers to reduce the spatial dimensions of the feature maps, which decreased computational complexity and made the model more robust to minor shifts and distortions in the input image.
- **End-to-End Learning:** It demonstrated that a complete system, from raw pixel input to final classification, could be trained as a single network.



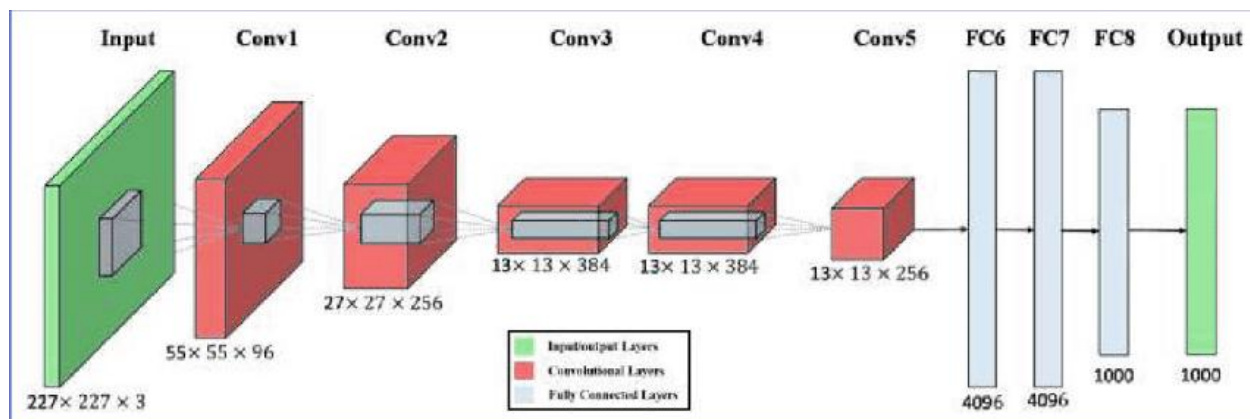
AlexNet

AlexNet burst onto the scene in 2012, winning the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) by a significant margin. Its success is widely considered the beginning of the deep learning revolution in computer vision.

- Layer 1: 96 filters of size $11 \times 11 \times 3$ (with a stride of 4)
- Layer 2: 256 filters of size 5×5
- Layer 3: 384 filters of size 3×3
- Layer 4: 384 filters of size 3×3
- Layer 5: 256 filters of size 3×3

Key Contributions:

- **Deep Architecture:** AlexNet was significantly deeper than previous networks, with 8 layers. This showed that stacking more layers could lead to much better performance.
- **ReLU Activation Function:** It was one of the first networks to use the **Rectified Linear Unit (ReLU)** as its activation function. ReLU helped to solve the **vanishing gradient problem**, allowing for more stable and faster training of deeper networks.
- **Dropout Regularization:** It introduced dropout to prevent overfitting, a common problem in deep neural networks with many parameters.
- **GPU Training:** The model was so large that it required two GPUs to train, highlighting the need for parallel processing in deep learning.



ZF-Net

ZFNet (Zeiler and Fergus Net), the winner of the 2013 ILSVRC, is an optimized refinement of the AlexNet architecture. Its defining characteristic is that its hyperparameters were chosen based on diagnostic feedback from a novel visualization technique, rather than raw intuition.

- **Deconvolutional Networks (Visualization):** ZFNet introduced the "DeconvNet," a mathematical method to map feature activations backward through the network into the original input pixel space. This allowed researchers to visually diagnose exactly what patterns (edges, textures, or complex objects) were triggering specific filters, marking a major step toward CNN interpretability.
- **Reduced Initial Filter Size:** By visualizing AlexNet, Zeiler and Fergus discovered that AlexNet's large 11×11 Layer 1 filters were failing to capture fine-grained details, resulting in "dead" or aliased features. ZFNet shrank the Layer 1 spatial filter size down to 7×7 .



VGG

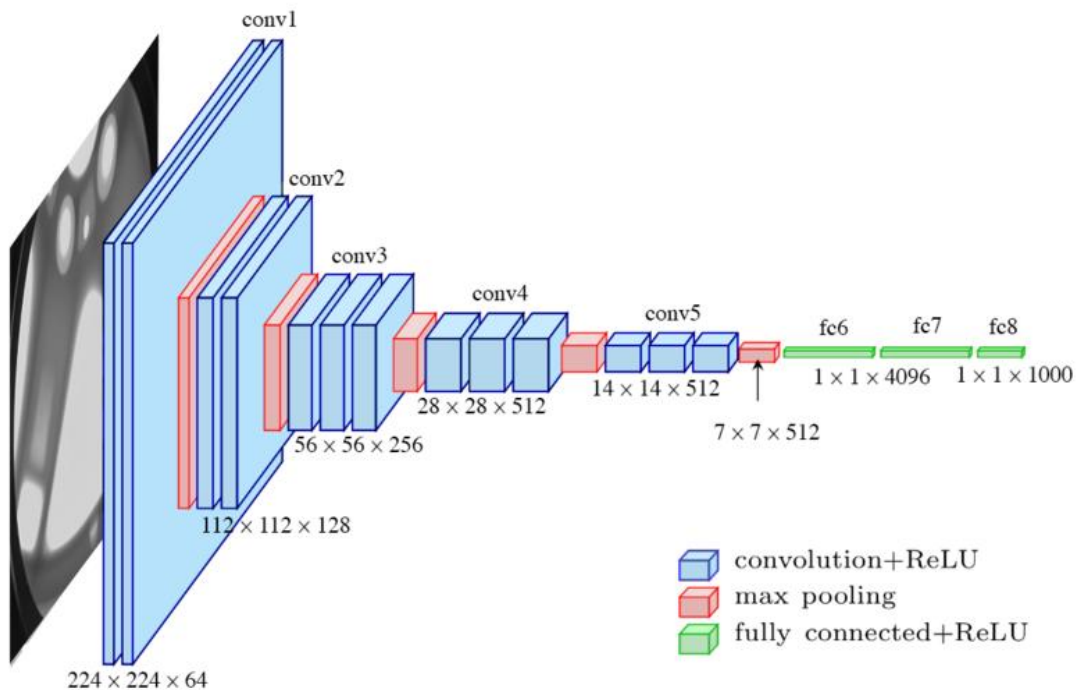
VGG (Visual Geometry Group) was introduced in 2014 and is famous for its simplicity and depth. Its main goal was to explore the effect of network depth on accuracy in a large-scale image recognition setting. VGGNet did not win 1st place in the main classification task of the 2014 ImageNet competition; it actually came in **2nd place**.

Geometrically, a stack of three 3×3 convolutional layers has the exact same "receptive field" (effective field of view) as a single 7×7 layer. Because there are three layers, the network applies three ReLU activation functions instead of just one, making the decision boundary significantly more discriminative.

$$R_L = R_{L-1} + (K_L - 1) \times S_{total}$$

Key Contributions:

- **Uniform Architecture:** VGG used a very simple and uniform architecture, consisting of a stack of small 3×3 convolutional filters. This showed that depth, rather than a variety of filter sizes, was a crucial component for high performance.
- **Increased Depth:** By using many small filters, VGG was able to build very deep networks (up to 19 layers) with a manageable number of parameters. This demonstrated that deep, uniform architectures could be highly effective.



GoogleNet

GoogleNet (also known as Inception) was a major winner of the ILSVRC 2014 competition. It was designed to be both deep and computationally efficient. 22-layer deep convolutional neural network.

Key Contributions:

- **Inception Module:** Its most important contribution was the **Inception module**, which allowed the network to learn multiple feature scales and representations simultaneously. It used 1×1 , 3×3 , and 5×5 convolutions along with a pooling layer in a single block. The intuition behind using multiple filter sizes like 3×3 and 5×5 simultaneously is to allow the network to "see" the image at different scales at the exact same time. Small filters (like 3×3) have a narrow receptive field, meaning they act like a magnifying glass to extract fine-grained, localized details such as sharp edges, textures, and tiny shapes. In contrast, larger filters (like 5×5) have a wider receptive field, allowing them to

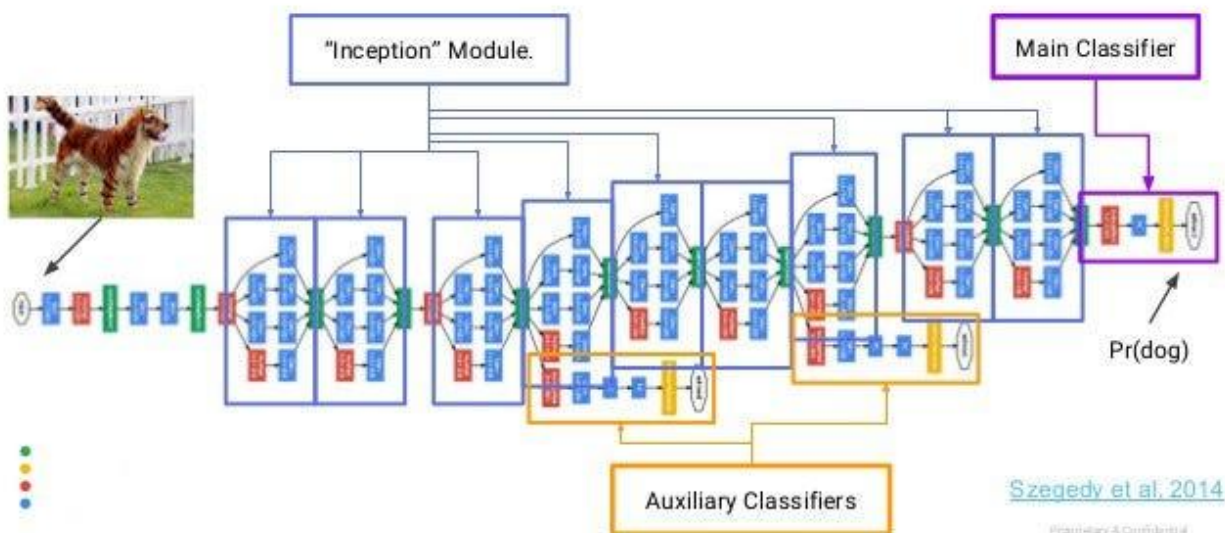
- aggregate information over a broader area to capture larger, global patterns and structural context
- **Global Average Pooling:** It replaced the fully connected layers at the end with a global average pooling layer, which drastically reduced the number of parameters and prevented overfitting. The original GoogLeNet (also known as Inception v1) contains approximately 6.8 million parameters compared to 138 million parameters in VGG.
- **Computational Efficiency:** The use of 1×1 convolutions (pointwise) for dimensionality reduction was a key trick to making the Inception module computationally cheaper, allowing the network to be much deeper and wider than its predecessors.

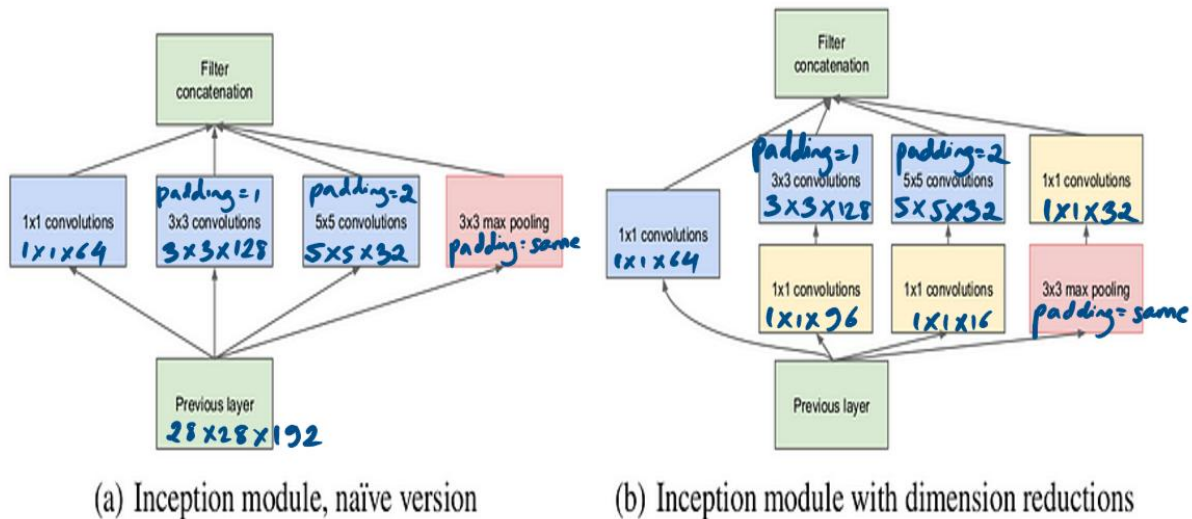
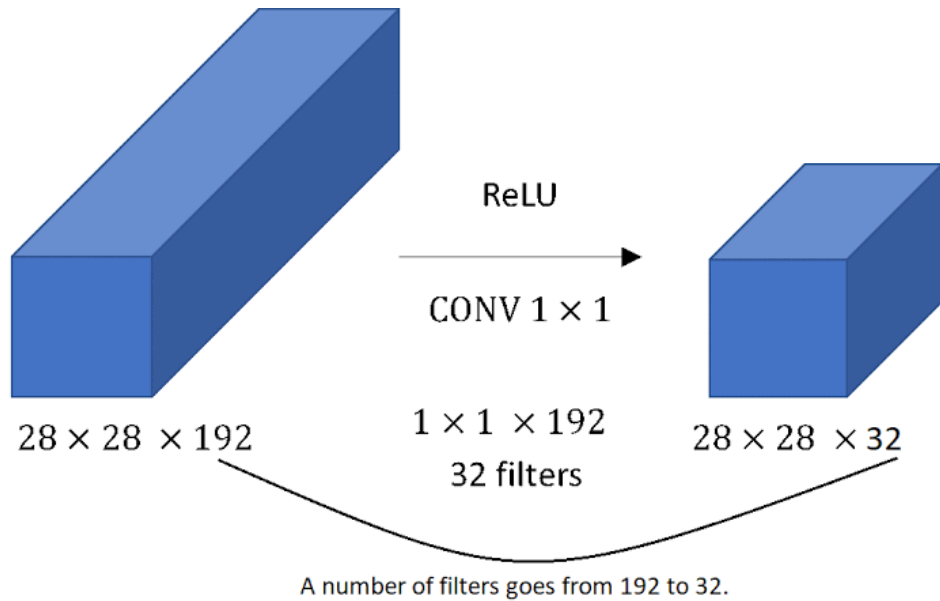
How It Works Mathematically

Assume the final convolutional block in a network outputs a feature map with dimensions $7 \times 7 \times 1024$ (Height $\times$ Width $\times$ Channels).

- For Channel 1, GAP takes all 49 values in that 7×7 grid, sums them up, and divides by 49 to output a single number.
- It repeats this exact process independently for all 1024 channels.
- **Output:** The 3D tensor is instantly collapsed from $7 \times 7 \times 1024$ into a $1 \times 1 \times 1024$ vector.

GoogLeNet (aka “Inception”) Architecture



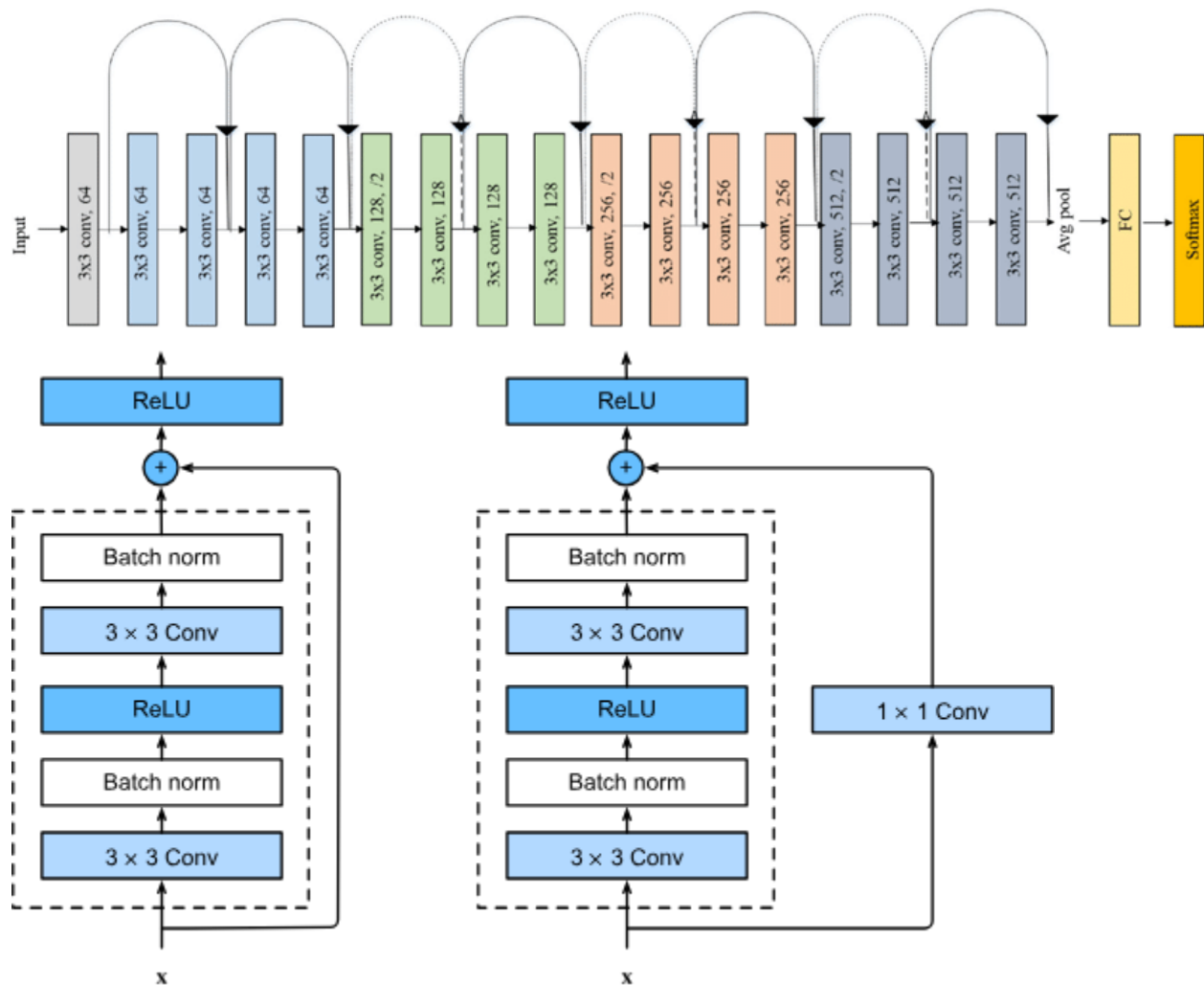


ResNet

ResNet (Residual Network) won the ILSVRC 2015 competition and introduced a fundamental concept that allowed for the training of extremely deep neural networks (up to 152 layers).

Key Contributions:

- **Residual Connections:** This is the most significant contribution of ResNet. The network uses "skip connections" or **residual connections** that allow the input of a layer to be added to its output. This helps to solve the **vanishing gradient problem** and allows the network to be much, much deeper without a significant performance drop.
- **Identity Mapping:** The skip connections essentially enable the network to learn an identity mapping, which is a crucial capability for deep networks. This means that if a layer finds that its output isn't a good fit, it can simply skip over itself and pass its input to the next layer.



Example 1: The Identity Block (Solid Arrow, Blue Stage)

This is a standard block in the middle of the blue stage. The goal is feature extraction without changing dimensions.

- Input x : $56 \times 56 \times 64$

Step 1: Main Path $\mathcal{F}(x)$

- Conv 1:** Applies 64 filters of size 3×3 with `stride=1` and `padding=1`.
 - Math:* Spatial size remains 56, channels remain 64.
 - Output:* $56 \times 56 \times 64$
- Conv 2:** Applies another 64 filters of size 3×3 with `stride=1` and `padding=1`.
 - Output:* $56 \times 56 \times 64$

Step 2: Shortcut Path

- Because this is an identity block, the original input x is routed completely unchanged
 - Output:* $56 \times 56 \times 64$

Step 3: Element-wise Addition

- The network adds the Main Path and the Shortcut Path together:

$$(56 \times 56 \times 64) + (56 \times 56 \times 64) = 56 \times 56 \times 64$$

- Final Block Output:** $56 \times 56 \times 64$

Example 2: The Downsampling Block (Dotted Arrow, Blue to Green Transition)

Now, the network needs to transition to the Green stage. It must halve the spatial dimensions to 28×28 and double the channels to 128.

- Input x :** $56 \times 56 \times 64$

Step 1: Main Path $\mathcal{F}(x)$

- Conv 1:** Applies 128 filters of size 3×3 with `stride=2` and `padding=1`.
 - Math (Spatial Downsampling):* $O = \lfloor (56 - 3 + 2(1))/2 \rfloor + 1 = 28$
 - Math (Channel Expansion):* 128 filters expand the depth to 128.
 - Output:* $28 \times 28 \times 128$

Step 2: Shortcut Path (The Problem and Solution)

- The Problem:* The original input x is $56 \times 56 \times 64$. We cannot add this to the Main Path's $28 \times 28 \times 128$ output. The dimensions clash.
- The Solution:* We pass x through a 1×1 **convolution** with 128 filters and `stride=2`.
 - Spatial downsampling:* The `stride=2` forces the 1×1 filter to skip every other pixel, shrinking 56×56 to 28×28 .
 - Channel expansion:* The 128 filters project the 64 channels up to 128.
 - Output:* $28 \times 28 \times 128$

Step 3: Element-wise Addition

- Now that the Shortcut matches the Main Path perfectly, they are added together:

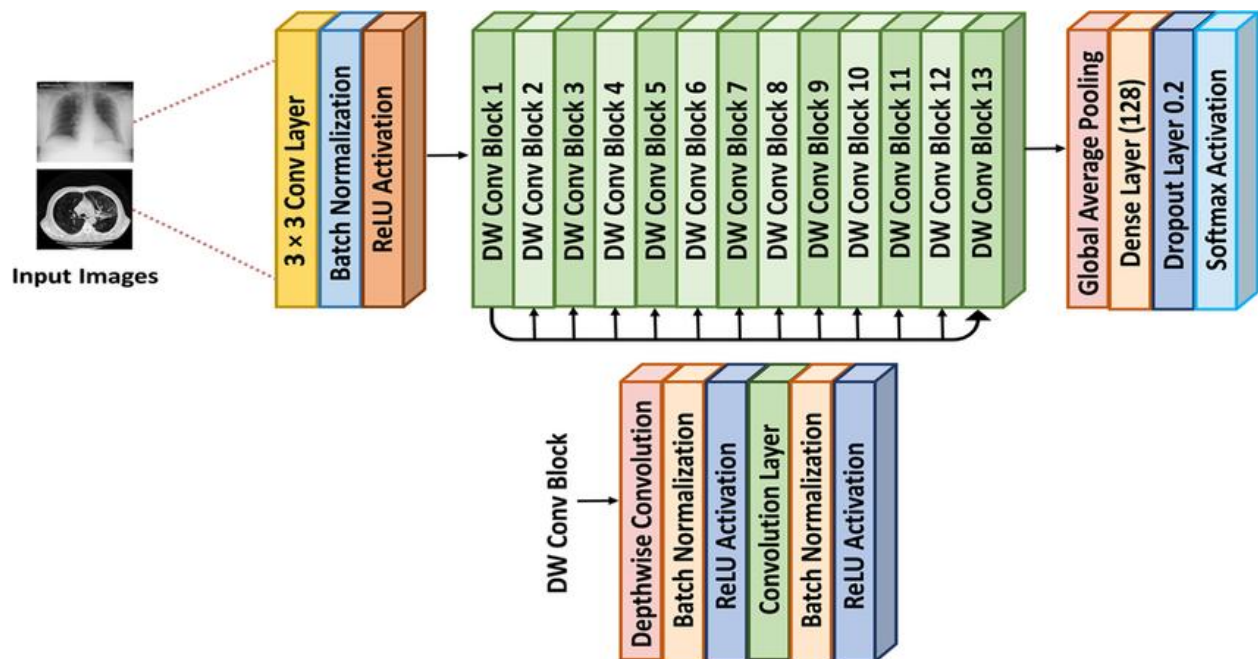
$$(28 \times 28 \times 128) + (28 \times 28 \times 128) = 28 \times 28 \times 128$$

MobileNet

MobileNet is a class of models designed specifically for mobile and embedded vision applications where computational resources and power are limited. 2017

Key Contributions:

- **Depthwise Separable Convolutions:** The core idea behind MobileNet is to replace a standard convolution with a depthwise separable convolution. This breaks a standard convolution into two smaller, more efficient operations: a depthwise convolution (applying a single filter to each input channel) and a pointwise convolution (a 1×1 convolution).
- **Computational Efficiency:** This technique significantly reduces the number of parameters to 4.2 million and computations, making the model much faster and smaller while maintaining a high level of accuracy.



The Scenario:

- **Input Feature Map:** $28 \times 28 \times 64$ (Height $\times$ Width $\times$ Channels)
- **Target Output:** $28 \times 28 \times 128$
- **Filter Size:** 3×3
- **Padding/Stride:** `padding='same' , stride=1`

Part 1: The Standard Convolution Baseline

In a standard convolution, every single 3×3 filter looks at all 64 input channels simultaneously. To get 128 output channels, we need 128 of these deep filters.

- **Filter Shape:** $3 \times 3 \times 64$
- **Number of Filters:** 128
- **Parameter Math:** $(3 \times 3 \times 64 \times 128) + 128$ (bias) = **73,856** parameters

Part 2: The Depthwise Separable Approach

Depthwise Separable Convolution breaks this exact same transformation into two distinct steps:

Step 1: Depthwise Convolution (Spatial Filtering Only)

Instead of looking at all 64 channels at once, we apply a single, flat 3×3 filter to **each channel individually**. Channel 1 gets its own filter, Channel 2 gets its own filter, and so on.

- **Filter Shape:** $3 \times 3 \times 1$
- **Number of Filters:** 64 (one for each input channel)
- **Parameter Math:** $(3 \times 3 \times 1 \times 64) + 64$ (bias) = **640** parameters
- **Intermediate Output:** $28 \times 28 \times 64$ (Spatial features are extracted, but the channels have not communicated with each other yet).

Step 2: Pointwise Convolution (Channel Mixing Only)

Now, we use our old friend—the 1×1 **convolution**—to mix the features across the 64 channels and expand the depth to our target of 128. The 1×1 filter looks at a single pixel but across all 64 channels.

- **Filter Shape:** $1 \times 1 \times 64$
- **Number of Filters:** 128
- **Parameter Math:** $(1 \times 1 \times 64 \times 128) + 128$ (bias) = **8,320** parameters
- **Final Output:** $28 \times 28 \times 128$

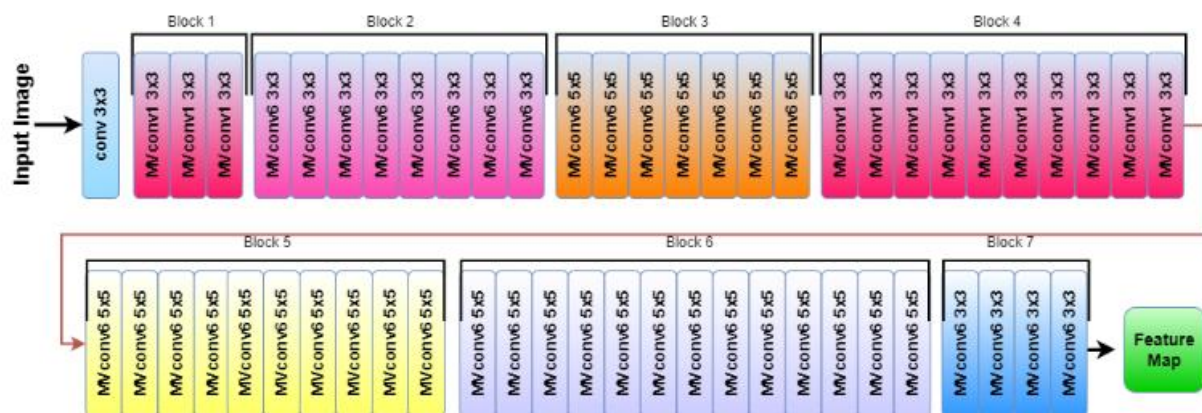
EfficientNet

EfficientNet is a recent architecture that systematically explores how to best scale a CNN. It introduces a new scaling method that uniformly scales the network's depth, width, and resolution. 2019

Key Contributions:

- **Compound Scaling:** EfficientNet's main contribution is its **compound scaling** method. It uses a compound coefficient to scale the network's depth, width, and image resolution simultaneously and in a principled way.
- **Optimized Performance:** This uniform scaling approach leads to a family of models that are much more computationally efficient and perform better than previous models, achieving state-of-the-art accuracy with significantly fewer parameters.

These architectures represent major milestones in the development of deep learning for computer vision, each contributing a novel idea that paved the way for the next generation of models.



Model Architecture	Number of Layers	Aprox Parameter Count	ImageNet Top-1 Accuracy	Design Intent
LeNet-5	5	~60 Thousand	N/A (<i>MNIST</i>)	Early foundational CNN designed for handwritten digit recognition (checks/ZIP codes).
AlexNet	8	60 Million	~63.3%	First deep CNN to win ImageNet; popularized the use of GPUs and the ReLU activation function.
VGG-16	16	138 Million	~71.3%	Demonstrated that stacking simple, uniform 3×3 convolutions yields highly accurate spatial hierarchies.
GoogLeNet (Inception v1)	22	6.8 Million	~69.8%	Designed to keep the computational budget low using parallel filters and 1×1 bottlenecks.
MobileNetV2	53	3.4 Million	~72.0%	Designed strictly for mobile and edge devices using depthwise separable convolutions.
ResNet-50	50	25.6 Million	~76.0%	Standard industrial workhorse using identity residual connections to solve vanishing gradients.
ResNet-152	152	60 Million	~78.3%	Deeper ResNet variant utilizing bottleneck blocks; requires heavy computation.
EfficientNet-B0	~82	5.3 Million	77.1%	Baseline version; beats GoogLeNet/MobileNet with a similar size using compound scaling.
EfficientNet-B7	~250+	66 Million	84.4%	The family's "max accuracy" heavy-lifter; dynamically scaled up in depth, width, and resolution.
GPipe (AmoebaNet)	~557	556 Million	~84.3%	Massive parameter giant that EfficientNet-B7 was designed to beat in computational efficiency.

Note: For EfficientNet models, "Number of Layers" is approximate, as they are constructed procedurally using compound scaling formulas ($d = \alpha^\phi$) rather than being manually stacked one by one.